

求最多可以派出多少支团队

题目描述

用数组代表每个人的能力，一个比赛活动要求参赛团队的最低能力值为N，每个团队可以由1人或者2人组成，且1个人只能参加1个团队，计算出最多可以派出多少只符合要求的团队。

输入描述

- 第一行代表总人数，范围1-500000
- 第二行数组代表每个人的能力
 - 数组大小，范围1-500000
 - 元素取值，范围1-500000
- 第三行数值为团队要求的最低能力值，范围1-500000

输出描述

最多可以派出的团队数量

用例

输入	5 3 1 5 7 9 8
输出	3
说明	说明 3、5组成一队 1、7一队 9自己一队 输出3

输入	7 3 1 5 7 9 2 6 8
输出	4
说明	3、5组成一队，1、7一队，9自己一队，2、6一队，输出4

输入	3 1 1 9 8
输出	1
说明	9自己一队，输出1

题目解析

本题要求最多的组队，而组队要求是：

- 可以1人组队，也可以2人组队
- 团队的能力值之和需要大于等于最低能力minCap要求

因此，为了组更多队伍，我们应该尽量让单人组队，即：

- 需要将能力值大于等于minCap的筛选出来，单人组队

然后剩余的人，按照能力值 **升序排序**，定义L,R指针，初始时L=0，R=k-1，k是剩余人总数

接着尝试L，R进行组队：

- 如果L，R两人的能力之和大于等于minCap，则组队成功，L++，R--
- 否则，说明L无法和任何人组队，因为R已经是当前最高能力的人，L无法和R组队，则也意味着无法和能力值比R低的人组队，因此L++

在上面过程中，计算出组队个数作为题解。

Java算法源码

```
1  import java.util.Arrays;
2  import java.util.Scanner;
3
4  public class Main {
5      public static void main(String[] args) {
6          Scanner sc = new Scanner(System.in);
7
8          int n = Integer.parseInt(sc.nextLine());
9
10         int[] capacities =
11             Arrays.stream(sc.nextLine().split(" ")).mapToInt(Integer::parseInt).toArray();
12
13         int minCap = Integer.parseInt(sc.nextLine());
14
15         System.out.println(getResult(n, capacities, minCap));
16     }
17
18     public static int getResult(int n, int[] capacities, int minCap) {
19         // 升序
20         Arrays.sort(capacities);
21
22         int l = 0;
23         int r = n - 1;
24
25         // 记录题解
26         int ans = 0;
27
28         // 单人组队
29         while (r >= l && capacities[r] >= minCap) {
30             r--;
31             ans++;
32         }
```

```

33 |
34 |     // 双人组队
35 |     while (l < r) {
36 |         int sum = capacities[l] + capacities[r];
37 |
38 |         // 如果两个人的能力值之和 $\geq minCap$ , 则组队
39 |         if (sum >= minCap) {
40 |             ans++;
41 |             l++;
42 |             r--;
43 |         } else {
44 |             // 否则将能力低的人剔除, 换下一个能力更高的人
45 |             l++;
46 |         }
47 |     }
48 |
49 |     return ans;
50 | }
51 |

```

JS算法源码

```

1  /* JavaScript Node ACM模式 控制台输入获取 */
2  const readline = require("readline");
3
4  const rl = readline.createInterface({
5      input: process.stdin,
6      output: process.stdout,
7  });
8
9  const lines = [];
10 rl.on("line", (line) => {
11     lines.push(line);
12
13     if (lines.length === 3) {

```

运行

```
14     let n = parseInt(lines[0]);
15     let capacities = lines[1].split(" ").slice(0, n).map(Number);
16     let minCap = parseInt(lines[2]);
17
18     console.log(getResult(n, capacities, minCap));
19
20     lines.length = 0;
21 }
22 });
23
24 function getResult(n, capacities, minCap) {
25     // 升序
26     capacities.sort((a, b) => a - b);
27
28     let l = 0;
29     let r = n - 1;
30
31     // 记录题解
32     let ans = 0;
33
34     // 单人组队
35     while (r >= l && capacities[r] >= minCap) {
36         r--;
37         ans++;
38     }
39
40     // 双人组队
41     while (l < r) {
42         const sum = capacities[l] + capacities[r];
43
44         // 如果两个人的能力值之和>=minCap, 则组队
45         if (sum >= minCap) {
46             ans++;
47             l++;
48             r--;
49         } else {
50             // 否则将能力低的人剔除, 换下一个能力更高的人
```

```
51 |         l++;52 |     }
53 | }
54 |
55 | return ans;
56 | }
```

Python算法源码

```
1 | # 输入获取
2 | n = int(input())
3 | capacities = list(map(int, input().split()))
4 | minCap = int(input())
5 |
6 |
7 | # 算法入口
8 | def getResult():
9 |     # 升序
10 |     capacities.sort()
11 |
12 |     l = 0
13 |     r = n - 1
14 |
15 |     # 记录题解
16 |     ans = 0
17 |
18 |     # 单人组队
19 |     while r >= l and capacities[r] >= minCap:
20 |         ans += 1
21 |         r -= 1
22 |
23 |     # 双人组队
24 |     while l < r:
25 |         total = capacities[l] + capacities[r]
```

```

26 | 27 |         # 如果两个人的能力值之和>=minCap, 则组队
28 |         if total >= minCap:
29 |             ans += 1
30 |             l += 1
31 |             r -= 1
32 |         else:
33 |             # 否则将能力低的人剔除, 换下一个能力更高的人
34 |             l += 1
35 |
36 |     return ans
37 |
38 |
39 | # 调用算法
40 | print(getResult())

```

C算法源码

```

1 | #include <stdio.h>
2 | #include <stdlib.h>
3 |
4 | #define MAX(a,b) ((a) > (b) ? (a) : (b))
5 |
6 | int cmp(const void* a, const void* b) {
7 |     return (*(int*) a) - (*(int*) b);
8 | }
9 |
10 | int main() {
11 |     int n;
12 |     scanf("%d", &n);
13 |
14 |     int capacities[n];
15 |     for(int i=0; i<n; i++) {
16 |         scanf("%d", &capacities[i]);
17 |     }
18 |

```

运行

```
19     int minCap;
20         scanf("%d", &minCap);
21
22     // 升序
23     qsort(capacities, n, sizeof(int), cmp);
24
25     int l = 0;
26     int r = n - 1;
27
28     // 记录题解
29     int ans = 0;
30
31     // 单人组队
32     while(r >= l && capacities[r] >= minCap) {
33         r--;
34         ans++;
35     }
36
37     // 双人组队
38     while(l < r) {
39         int sum = capacities[l] + capacities[r];
40
41         if(sum >= minCap) {
42             // 如果两个人的能力值之和 $\geq minCap$ , 则组队
43             ans++;
44             l++;
45             r--;
46         } else {
47             // 否则将能力低的人剔除, 换下一个能力更高的人
48             l++;
49         }
50     }
51
52     printf("%d\n", ans);
53
54     return 0;
55 }
```


C++ 算法源码

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      int n;
6      cin >> n;
7
8      vector<int> capacities;
9      for (int i = 0; i < n; i++) {
10         int cap;
11         cin >> cap;
12
13         capacities.emplace_back(cap);
14     }
15
16     int minCap;
17     cin >> minCap;
18
19     // 升序
20     sort(capacities.begin(), capacities.end());
21
22     int l = 0;
23     int r = n - 1;
24
25     // 记录题解
26     int ans = 0;
27
28     // 单人组队
29     while (r >= l && capacities[r] >= minCap) {
30         r--;
31         ans++;
```

```
32 |     }  
33 |  
34 | // 双人组队  
35 | while (l < r) {  
36 |     int sum = capacities[l] + capacities[r];  
37 |  
38 |     // 如果两个人的能力值之和 $\geq minCap$ , 则组队  
39 |     if (sum >= minCap) {  
40 |         ans++;  
41 |         l++;  
42 |         r--;  
43 |     } else {  
44 |         // 否则将能力低的人剔除, 换下一个能力更高的人  
45 |         l++;  
46 |     }  
47 | }  
48 |  
49 | cout << ans << endl;  
50 |  
51 | return 0;  
52 | }
```